

ALU- Design Module:

```
module ALU(zero, result, control, A, B);
```

```
input [31:0] A, B;
```

```
input [2:0] control;
```

```
output reg zero;
```

```
output reg [31:0] result;
```

```
always @(*)
```

```
begin
```

```
case (control)
```

```
3'b000 : result = A+B;
```

```
3'b001 : result = A-B;
```

```
3'b101 : result = A<B;
```

```
3'b011 : result = A|B;
```

```
3'b010 : result = A&B;
```

```
default : result = 32'bx;
```

```
endcase
```

```
end
```

```
always @ (result)
```

```
begin
```

```
if (result)
```

```
zero = 0;
```

```
else
```

```
zero = 1;
```

```
end
```

```
endmodule
```

Test Bench:

```
module tb(); reg [31:0] A, B; reg [2:0] control;
```

```
wire zero;
```

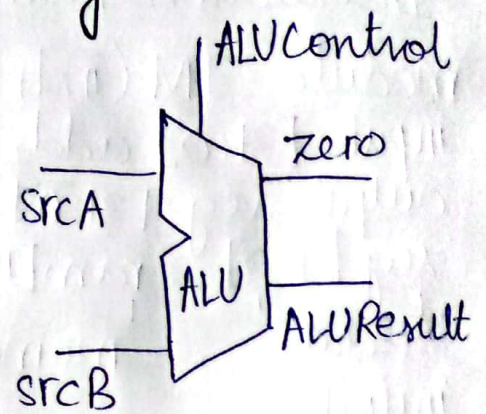
```
wire [31:0] result;
```

```
ALU uut (zero, result, control, A, B);
```

```
initial begin
```

```
A = 3; B = 4; control = 3'b000; # 5
```

```
endmodule
```



or

```
if (control == 3'b000)
```

```
begin
```

```
result = A+B;
```

```
end
```

```
elseif (control == 3'b010)
```

```
begin
```

```
result = A&B;
```

```
end
```

```
else
```

```
begin
```

```
result = 32'bx;
```

```
end
```

Assign zero = (result == 0)? 1'b1: 1'b0;

Instruction Memory:

```
module IM(instr, add);  
input [31:0] add;  
output reg [31:0] instr;  
reg [7:0] data;      ram[15:0];  
                      memory location
```

initial

begin

ram[0] = 8'b0000000000000000;

ram[1] =

ram[15] = 8'b01110101;

end

always @ (add)

instr = { ram[add+3], ram[add+2], ram[add+1], ram[add] };

endmodule

Testbench:

```
module IM_tb();
```

```
reg [31:0] add;
```

```
wire [31:0] instr;
```

```
IM uut (instr, add);
```

```
initial begin
```

```
add = 0;
```

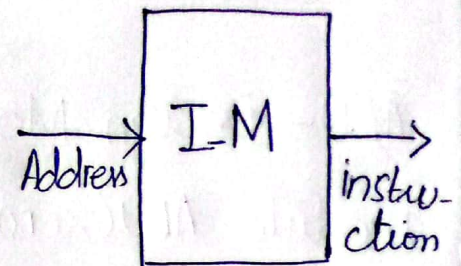
```
add = 4; #10
```

```
add = 8; #10
```

```
add = 12; #10
```

```
end
```

```
endmodule
```



Data Memory:

```
module DM (Read, write, add, clk, mem), mem-add
```

```
input [31:0] add, write;
```

```
input clk, mem;
```

```
reg [7:0] i; ram [63:0];
```

```
output [31:0] Read;
```

```
integer i;
```

```
initial
```

```
begin
```

```
for (i=0; i<64; i=i+1)
```

```
    ram[i]=i;
```

```
end
```

```
assign Read = {ram[add+3], ram[add+2],  
               ram[add+1], ram[add]};
```

```
always @(posedge clk) // write operation happen  
begin // on +ve edge of clock
```

```
if (mem)
```

```
    assign write = {ram[add+3], ram[add+2],  
                   ram[add+1], ram[add]};
```

```
end
```

```
endmodule
```

Test Bench:

```
module tb();
```

```
reg [31:0] add, write;
```

```
reg clk, mem;
```

```
wire [31:0] Read;
```

```
DM uut (Read, write, add, clk, mem);
```

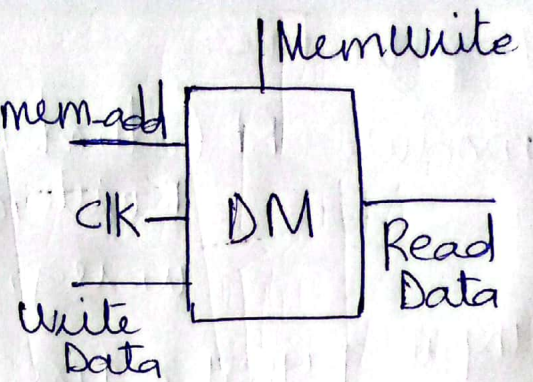
```
always #5clk = ~clk
```

```
initial begin clk=0;
```

```
A=0; mem=0; write = 32'hff; #10
```

```
end
```

```
endmodule
```



Register File:

```
module RF( $\delta d1, \delta d2, \delta s1, \delta s2,$ 
           $r\_write, clk, \cancel{w\_data}$ 
           $w\_reg, w\_data$ );
```

```
input [4:0]  $\delta s1, \delta s2, w\_reg$ ;
```

```
input  $r\_write, clk$ ;
```

```
output [31:0]  $\delta d1, \delta d2$ ;
```

```
input [31:0]  $w\_data$ ;
```

```
integer i;
```

```
always @ (posedge  $clk$ )
```

```
if ( $\delta r\_write$ )
```

```
begin
```

```
 $\delta f[w\_reg] = w\_data$ ;
```

```
end
```

```
initial
```

```
begin
```

```
for ( $i=0$ ;  $i < 32$ ;  $i++$ )
```

```
begin
```

```
 $\delta f[i] = i$ ;
```

```
end
```

```
end
```

```
endmodule
```

```
assign  $\delta d1 = \delta f[\delta s1]$ ;
```

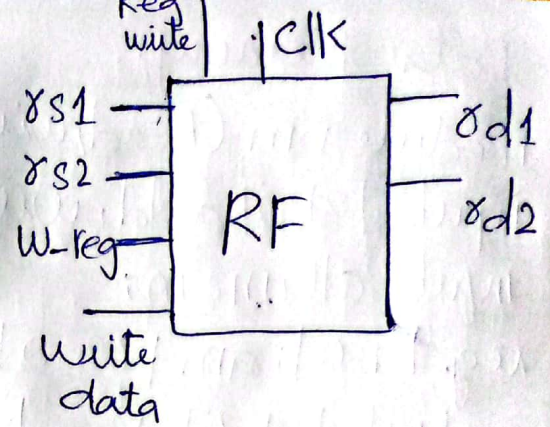
```
assign  $\delta d2 = \delta f[\delta s2]$ ;
```

```
endmodule
```

Test bench:

```
 $\delta s1 = 3$ ;  $\delta s2 = 2$ ;  $w\_reg = 6$ ;  $regwrite = 0$ ; or 1
```

```
 $w\_data = 32'h44$ ; #10
```



```
reg[31:0]  $\delta f[31:0]$ ;
```


Immediate Extender:

```
module IE(Ex, Src, Instr);  
input [31:0] Instr;  
input [1:0] Src;  
output [31:0] Ex;
```

```
always @ (Instr or Src)  
begin
```

```
case (Src)
```

```
2'b00 : Ex = {{20{Instr[31]}}, {Instr[31:20]}};
```

```
2'b01 : Ex = // given instruction
```

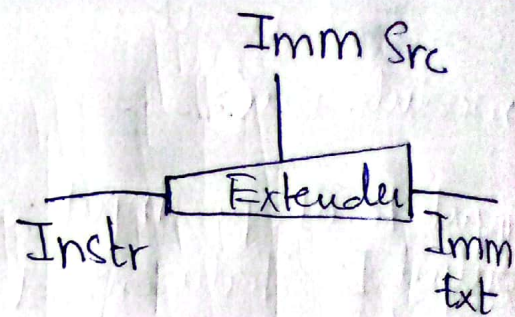
```
2'b10 : Ex =
```

```
default : Ex = 32'bx;
```

```
endcase
```

```
end
```

```
endmodule
```



Testbench:

```
Instr = 32'h A45;
```

```
Src = 2'b01
```

```
#10
```

```
Instr = 32'h 002;
```

```
Src = 2'b00;
```

```
#10
```


Control unit :

module cu(

input [6:0] op; input zero, f7; input [2:0] f3; op

output [2:0] alucontrol;

output [1:0] imm-src;

output pcsrc, resultsrc, m-w, r-w, alu-src;

wire branch;

wire alu-op;

md (branch, r-src, m-w, alu-src,
imm-src, r-w, op);

ad (alucontrol, alu-op, f7, f3);

assign pc-src = zero & branch;

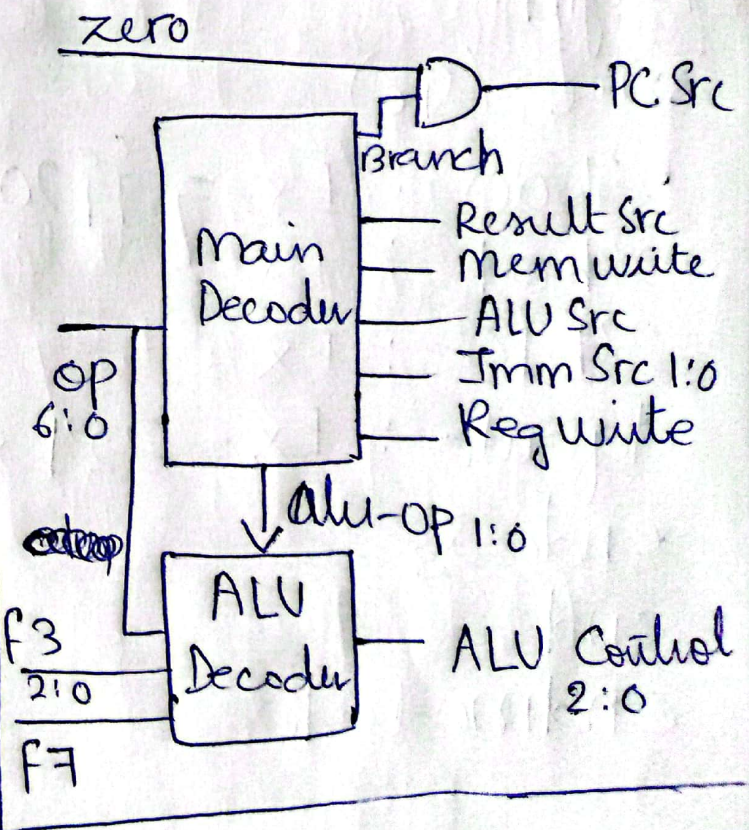
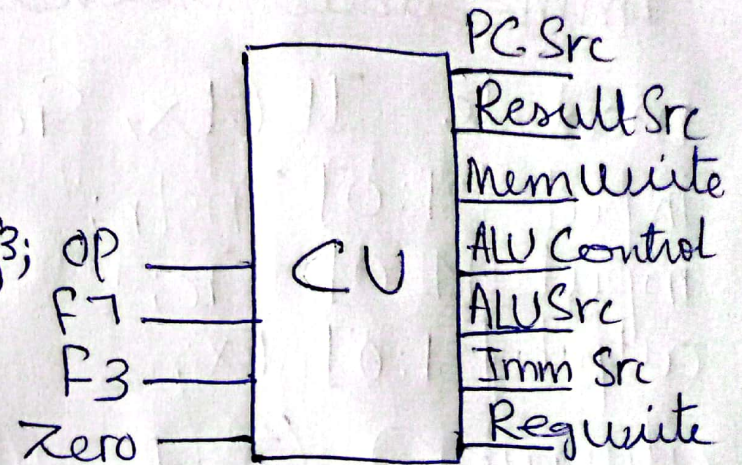
end module.

module ad(alucontrol, alu-op, f7, f3);

input [1:0] alu-op; input [2:0] f3;

input f7; output [3:0] alucontrol;

wire [3:0]



ALU Decoder:

```
module AD(alu-control, alu-op,
          f3, f7);
  input [2:0] f3;
  input [6:0] f7;
  input [1:0] alu-op;
  output reg [2:0] alu-control;
  always @ (*)
```

```
begin
```

```
  case (alu-op)
```

```
    2'b00: alu-control = 3'b000;
```

```
    2'b01: begin
```

```
      case (f3)
```

```
        3'b000: alu-control = 001;
```

```
        3'b001: alu-control = 100;
```

```
        default: alu-control = 3'bxxx;
```

```
      endcase
```

```
    end
```

```
    2'b10: begin
```

```
      case (f3)
```

```
        3'b000: begin
```

```
          7'b0000000, 7'b0000001, 7'b0000010 : alu-control = 3'b000;
```

```
          7'b0000011 : alu-control = 3'b001;
```

```
          default: alu-control = 3'bxxx;
```

```
        endcase
```

```
      end
```

```
        3'b010: alu-control = 101;
```

```
        3'b110: alu-control = 011;
```

```
        3'b111: alu-control = 010;
```

```
      endcase default: alu-control = 3'bxxx;
```

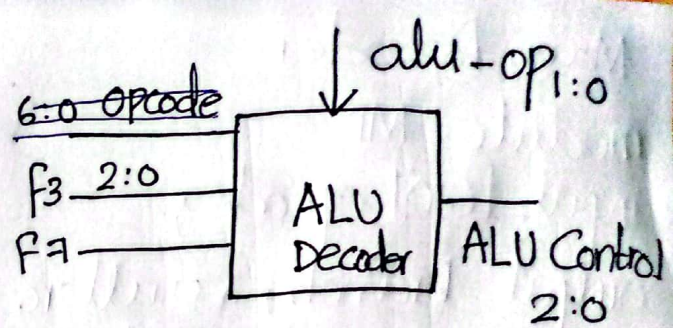
```
    end
```

```
  default: alu-control = 3'bxxx;
```

```
  endcase
```

```
end
```

```
endmodule
```



ALUop	F3	F7	ALUControl
00	x	x	000
01	000	x	001
	001	x	100
10	000	00,01,10	000
	000	11	001
	010	x	101
	110	x	011
	111	x	010

Main Decoder:

```
module MD
```

```
input [6:0] opcode;
```

```
output branch, resultsrc, immsrc, alusrc, m_write,  
r_write, aluop;
```

```
output [1:0] immsrc, aluop;
```

```
wire [8:0] controls;
```

```
assign {alusrc, resultsrc, immsrc, r_write,  
m_write, branch, aluop} = controls;
```

```
assign control = (opcode == 7'b00000011) ? 9'b100101000 :  
(opcode == 7'b01000011) ? 9'b00111x000 :  
(opcode == 7'b01100011) ? 9'b1xx000010 :  
(opcode == 7'b11000011) ? 9'b01000x101 :  
9'bxxxxxxxxxx;
```

```
end module
```

Op	regwrite	ImmSrc	ALUSrc	Memwrite	Result Src	Branch	ALU Op
00000011	1	00	1	0	1	0	00
01000011	0	01	1	1	X	0	00
01100011	1	xx	0	0	0	0	10
11000011	0	10	0	0	X	1	01

